Notes – Software Description

Oct 21, 2026 · @Shreya Sharma

Overview

Notes is a shared notes board that runs on Kubernetes. Users open it in a web browser, add short text notes, see every note, and delete notes.

The application is built from two microservices I developed myself in Node.js, plus a MongoDB database running as a third service. Each microservice provides its own REST API, and one microservice consumes the other's API over the network.

What it does

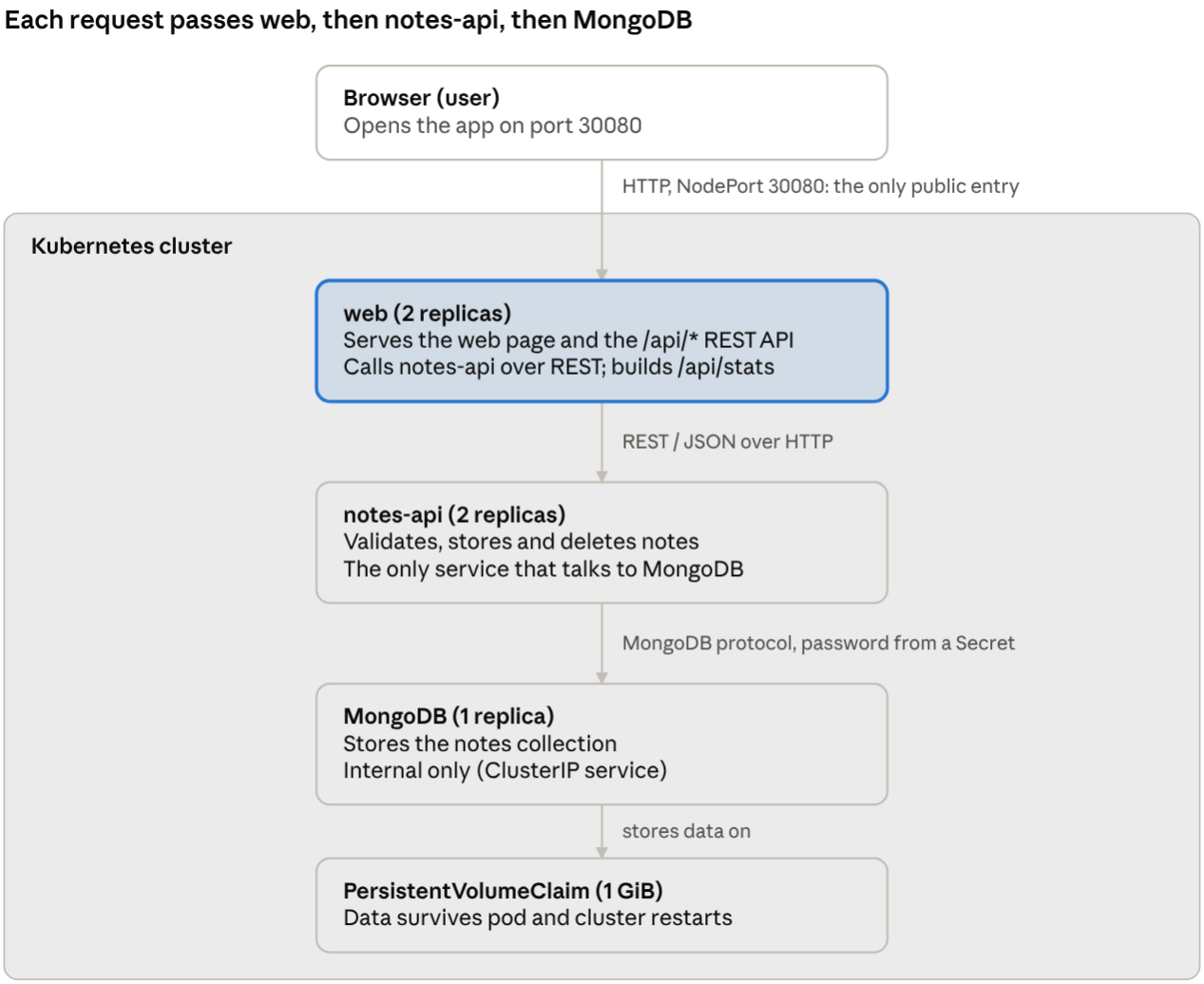
From the web page, a user can:

- **Add a note** of 1 to 500 characters. Empty or longer notes are rejected with an error message.
- **See all notes** stored in the database, shared between all users.
- **Delete a note** with the button next to it.
- **See statistics** in the page footer: the number of notes and their average length.
- **See which pods served the request.** The footer shows the name of the `web` pod and the `notes-api` pod that answered. Refreshing the page shows the names change as Kubernetes spreads requests across replicas.

Notes are stored on persistent storage, so they are still there after the database pod, the deployment or the cluster restarts.

How it is built

The browser only ever talks to `web`. `web` holds no data: it forwards each request to `notes-api` over REST and returns the answer, and only `notes-api` reads and writes MongoDB.



request flow · browser, 2 microservices, database

REST APIs

Both microservices exchange JSON over HTTP.

web (public, used by the browser)

Method	Path	What it does
GET	`/`	Returns the web page (HTML + JavaScript)
GET	`/api/notes`	Lists all notes (calls notes-api)
POST	`/api/notes`	Adds a note; body `{"text": "..."}` (calls notes-api)
DELETE	`/api/notes/{id}`	Deletes a note (calls notes-api)

Method	Path	What it does
GET	`/api/stats`	Returns note count, average length and the serving pod names
GET	`/health`	Health check used by Kubernetes

notes-api (internal, reachable only inside the cluster)

Method	Path	What it does
GET	`/notes`	Returns all notes from MongoDB
POST	`/notes`	Validates the text (1–500 characters) and stores the note; returns `201`
DELETE	`/notes/{id}`	Deletes a note; `400` for an invalid id, `404` if not found
GET	`/health`	Health check used by Kubernetes

If notes-api cannot be reached within 3 seconds, `web` answers `503 notes-api unavailable` instead of hanging.

Running on Kubernetes

The whole application is deployed with one manifest, `k8s/app.yaml`, using `kubectl apply -f k8s/app.yaml`.

- **Images:** `notes-api` and `notes-web` are built from their Dockerfiles and pushed to Docker Hub, where Kubernetes pulls them. MongoDB uses the official `mongo:7` image.
- **Access from outside:** only `web` is exposed, as a NodePort service on port 30080 (e.g. `http://localhost:30080` or `http://<server-ip>:30080`). `notes-api` and `mongo` are internal ClusterIP services.
- **Independent scaling:** `web` and `notes-api` are separate Deployments, each starting with 2 replicas. Either can be scaled on its own, e.g. `kubectl scale deployment notes-api --replicas=4`.
- **Persistent database:** MongoDB runs as a single replica and stores its data on a 1 GiB PersistentVolumeClaim.
- **Credentials:** the database username and password come from a Kubernetes Secret, not from the code.
- **Health checks:** each microservice has a `/health` endpoint used as a readiness probe, so traffic only goes to pods that are ready.

Scale assumption

The application is deliberately small, and a notes board with a few users does not need to scale.

To see why the design makes sense, imagine it is a company-wide notice board used by thousands of employees. Page and read traffic would then be much higher than writes. You could add more `web` replicas to handle browser traffic, or more `notes-api` replicas to handle data requests, without changing the other service.

Technologies used

Part	Technology
Microservices	Node.js 20, Express 5
REST client in `web`	Node.js built-in `fetch`
Database	MongoDB 7, official `mongodb` Node.js driver
User interface	Plain HTML and JavaScript
Containers	Docker (`node:20-alpine` base image), images on Docker Hub
Orchestration	Kubernetes: Deployments, Services, Secret, PersistentVolumeClaim